Лабораторная работа №6. Сетевое взаимодействие контейнеров

Тема: Docker bridge, DNS и связь Express API с PostgreSQL

Цель: Создать пользовательскую Docker-сеть, подключить Express API к PostgreSQL и научиться диагностировать и восстанавливать сетевое взаимодействие контейнеров.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №6. Сетевое взаимодействие контейнеров. Продолжительность — 4 академических часа (180 минут). Среда — Debian 13, Docker Engine, Node.js/Express, PostgreSQL 17. Эта работа продолжает проект предыдущих лабораторных. Docker Compose ещё не используется.

2. Тема и цель

Тема: создание пользовательской bridge-сети Docker и организация взаимодействия контейнеров Express API и PostgreSQL. Цель: научиться создавать сети, подключать существующие контейнеры, использовать DNS-имя вместо IP-адреса, публиковать только порт HTTP API и выполнять диагностику отказов.

3. Планируемые результаты

1. Создать и исследовать пользовательскую bridge-сеть course-net.
2. Подключить существующий контейнер PostgreSQL без удаления данных.
3. Собрать образ Express API с драйвером pg и маршрутом проверки БД.
4. Проверить соединение по имени pg-course через встроенный DNS Docker.
5. Отличать сетевую ошибку от ошибки доступа к PostgreSQL.
6. Безопасно восстановить сервис после диагностического эксперимента.

4. Оборудование и исходные данные

Нужны Linux-ВМ Debian 13 с установленным Docker Engine, терминал, Node.js и npm для подготовки исходного кода (либо подготовленные преподавателем файлы), интернет для загрузки образов. Команды Docker выполняйте через sudo. Если контейнер pg-course и том pgdata-course созданы в ЛР №5, не удаляйте их: здесь они повторно используются.

Параметры сквозного проекта

Параметр	Значение
Сеть	course-net (user-defined bridge)
PostgreSQL-контейнер	pg-course
Образ БД	postgres:17
Том БД	pgdata-course
База данных	shop
Пользователь	student
Пароль учебной БД	Lab5-ChangeMe-2026

Параметр	Значение
Express-контейнер	api-net6
Образ API	express-pg:net6
HTTP API	127.0.0.1:3000 → 3000
Адрес БД из API	pg-course:5432

Пароль указан только для изолированной учебной виртуальной машины. Не используйте его в рабочих системах и не публикуйте собственные пароли в отчётах. Если в ЛР №5 пароль был изменён, используйте реальный пароль созданного кластера PostgreSQL.

5. Топология

Связь контейнеров

```
Браузер / curl (Debian)
        | GET http://127.0.0.1:3000/api/db-check
        | опубликованный порт 127.0.0.1:3000
        v
     [api-net6:3000]  -- pg-course:5432 --> [pg-course:5432]
            \__________ course-net ____________/
                                        |
                                   pgdata-course
```

Docker DNS переводит имя pg-course в адрес контейнера PostgreSQL. Имена и адреса интерфейсов Linux на виртуальных машинах могут различаться, поэтому не прописывайте IP контейнера вручную. В этой лабораторной соединяются контейнеры на одном Docker-хосте.

6. Подготовка и проверка среды

Этап 1. Проверить Docker и PostgreSQL

Проверка Docker и объектов ЛР №5

```
sudo docker version
sudo docker ps -a
sudo docker volume ls
```

Убедитесь, что есть контейнер pg-course и том pgdata-course. Если контейнер отсутствует, не создавайте пустой новый кластер поверх неизвестного тома без согласования с преподавателем: сначала выясните, почему исчез объект ЛР №5.

Запуск существующего PostgreSQL

```
sudo docker start pg-course
sudo docker exec pg-course pg_isready -U student -d shop
```

Если контейнер был остановлен, start его запустит. Команда pg_isready должна сообщить accepting connections после инициализации БД. При ошибке смотрите sudo docker logs pg-course. Если контейнер уже работает, start не нужен.

Проверка ранее сохранённых данных

```
sudo docker exec pg-course psql -U student -d shop -c "SELECT id, name FROM categories
ORDER BY id;"
```

Ожидается таблица categories из ЛР №5. Эта проверка подтверждает, что мы работаем с прежним кластером PostgreSQL и томом, а не с новой пустой БД. При отсутствии таблицы сообщите преподавателю.

7. Создание и исследование Docker-сети

Этап 2. Создать сеть course-net

Существующие сети

```
sudo docker network ls
```

Найдите стандартные bridge, host, none и пользовательские сети. Сеть course-net далее создаётся один раз.

Создать пользовательскую bridge-сеть

```
sudo docker network create --driver bridge course-net
```

Флаг --driver bridge задаёт тип; course-net — имя сети. Если сеть уже существует, повторно её создавать не нужно. Проверка: сеть присутствует в docker network ls.

Исследовать сеть

```
sudo docker network inspect course-net
```

Найдите Driver, Subnet, Gateway и Containers. IP и шлюз назначаются Docker автоматически и могут отличаться на каждом ПК.

Этап 3. Подключить PostgreSQL

Подключение и проверка

```
sudo docker network connect course-net pg-course
sudo docker network inspect course-net
```

Подключаем работающий контейнер к дополнительной сети без пересоздания и без затрагивания тома. Если команда возвращает ошибку already exists, значит контейнер уже подключён. В Containers должен отображаться pg-course.

Проверка DNS из другого контейнера

```
sudo docker run --rm --network course-net busybox:1.36 nslookup pg-course
```

BusyBox — временный диагностический контейнер. Ожидается успешное определение адреса pg-course. Команда --rm удаляет только временный BusyBox после завершения. При ошибке проверьте сеть через inspect.

8. Подготовка приложения Express API

Создайте отдельный каталог express-net6. Это небольшой диагностический API для проверки реального соединения с PostgreSQL. Он не заменяет исходный проект товаров из ЛР №4; позднее аналогичный модуль подключения можно перенести в него. Основная задача работы — сеть, поэтому код сервера предоставляется готовым.

Этап 4. Создать проект

Создание каталога и зависимостей

```
mkdir -p ~/express-net6
cd ~/express-net6
npm init -y
npm install express pg
npm pkg set type=module
```

npm создаёт package.json и package-lock.json, устанавливает Express и драйвер PostgreSQL pg. Для сборки используется npm ci и lock-файл. Если Node.js/npm отсутствуют на учебной машине, получите подготовленную папку проекта у преподавателя.

Файл app.js

```
import express from "express";
import pg from "pg";

const { Pool } = pg;
const app = express();

const pool = new Pool({
  host: process.env.PGHOST,
  port: 5432,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD,
  database: process.env.PGDATABASE,
  connectionTimeoutMillis: 3000
});

app.get("/api/db-check", async (req, res) => {
  try {
    const result = await pool.query("SELECT COUNT(*)::int AS total FROM categories");
    res.json({ ok: true, categories: result.rows[0].total });
  } catch (error) {
    console.error("DB connection error:", error.message);
    res.status(503).json({ message: "База данных недоступна" });
  }
});

app.listen(3000, "0.0.0.0", () => {
  console.log("API слушает порт 3000");
});
```

Pool подключается к PostgreSQL по параметру PGHOST. Маршрут /api/db-check выполняет SQL-запрос к таблице categories, созданной в ЛР №5. В случае сетевого отказа или ошибки

БД HTTP API отвечает 503. Сервер слушает 0.0.0.0 внутри контейнера, чтобы быть доступным через Docker -p.

Файл Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY app.js ./
EXPOSE 3000
CMD ["node", "app.js"]
```

FROM задаёт Node.js; WORKDIR — каталог; COPY сначала переносит метаданные npm для кэширования; RUN устанавливает зависимости. EXPOSE документирует порт, но не публикует его на хост.

Файл .dockerignore

```
node_modules
.git
.env
npm-debug.log
```

Файл исключает локальные зависимости и секреты из контекста сборки. В Dockerfile нельзя записывать реальные рабочие пароли.

9. Сборка и запуск приложения

Этап 5. Собрать Docker-образ

Сборка образа Express API

```
cd ~/express-net6
sudo docker build -t express-pg:net6 .
sudo docker image ls express-pg
```

Точка в конце означает текущий контекст сборки. Ожидается локальный образ express-pg с тегом net6. При ошибке npm ci убедитесь в наличии package-lock.json и правильных зависимостей.

Этап 6. Запустить Express в общей сети

Запуск API с переменными подключения

```
sudo docker run -d --name api-net6 --network course-net -p 127.0.0.1:3000:3000 -e
PGHOST=pg-course -e PGUSER=student -e PGPASSWORD=Lab5-ChangeMe-2026 -e PGDATABASE=shop
express-pg:net6
```

Контейнер api-net6 подключён к course-net и видит pg-course по DNS-имени. Порт 3000 доступен только на loopback Debian-хоста. Пароль указан для учебного примера; командная строка и параметры Docker могут быть доступны администраторам, поэтому в производственной системе такой способ передачи секрета требует замены.

Проверка состояния

```
sudo docker ps
sudo docker network inspect course-net
sudo docker logs api-net6
```

Ожидается состояние Up обоих контейнеров, сеть course-net содержит api-net6 и pg-course. В журнале API должно быть сообщение о запуске HTTP-сервера.

Этап 7. Проверить HTTP-запрос и SQL

```
HTTP-запрос с Debian-хоста

curl -i http://127.0.0.1:3000/api/db-check
```

Ожидается HTTP/1.1 200 OK и JSON вида {"ok":true,"categories":2} при наличии двух записей. Число categories зависит от реальных данных, оставшихся после ЛР №5. Если 503, проверьте сеть и PostgreSQL; если соединения нет, проверьте порт и логи API.

```
Проверка количества строк напрямую в БД

sudo docker exec pg-course psql -U student -d shop -c "SELECT COUNT(*) FROM categories;"
```

Сравните число строк в psql и число categories в ответе API: они должны совпасть. Это доказывает, что Express не имитирует результат, а действительно обращается к SQL-таблице.

10. Диагностические эксперименты

Эксперимент A. Отключение от сети

Сначала проверьте успешный запрос curl. Затем отсоедините только API-контейнер от course-net. Сам API останется работать, но не сможет использовать DNS-имя pg-course через эту сеть.

```
Намеренно нарушить связь

sudo docker network disconnect course-net api-net6
curl -i http://127.0.0.1:3000/api/db-check
sudo docker logs api-net6
```

Ожидается ошибка SQL-соединения и HTTP 503, если HTTP-сервер продолжает работать. Ошибка может содержать ENOTFOUND или другую диагностическую информацию. Нельзя заранее гарантировать один конкретный код: он зависит от состояния DNS/сети.

```
Восстановить связь

sudo docker network connect course-net api-net6
curl -i http://127.0.0.1:3000/api/db-check
```

После подключения обратно маршрут должен вернуть HTTP 200 при работающей БД. Если сразу после повторного подключения ответ ещё не успешный, проверьте логи и повторите после короткого ожидания.

Эксперимент B. Остановка PostgreSQL

```
Намеренно остановить СУБД

sudo docker stop pg-course
curl -i http://127.0.0.1:3000/api/db-check
```

Ожидается HTTP 503: API доступен, но SQL-запрос не может выполняться. Том pgdata-course не удаляется. Зафиксируйте диагностический результат.

```
Восстановить PostgreSQL

sudo docker start pg-course
sudo docker exec pg-course pg_isready -U student -d shop
curl -i http://127.0.0.1:3000/api/db-check
```

Дождитесь accepting connections. При успешном восстановлении API снова возвращает HTTP 200 с прежним количеством категорий. Если БД стартует дольше, повторите pg_isready.

11. Самостоятельное задание

7. Выполните docker network inspect course-net и найдите подключённые контейнеры.
8. Выполните docker exec pg-course psql -U student -d shop -c "SELECT COUNT(*) FROM categories;".
9. Выполните GET /api/db-check и сравните значения.
10. Объясните, почему в PGHOST применяется pg-course, а не localhost.
11. Объясните, почему мы опубликовали 3000, но не публиковали 5432.
12. Сохраните в отчёт скриншот результатов диагностики и схему сети.

12. Итоговая проверка

13. Контейнеры pg-course и api-net6 запущены.
14. Оба находятся в сети course-net.
15. API доступен по http://127.0.0.1:3000/api/db-check.
16. В ответе указаны ok: true и реальное количество строк таблицы categories.
17. После диагностических экспериментов связь восстановлена.
18. Том pgdata-course сохранён, записи из ЛР №5 не удалены.

13. Содержание отчёта

19. Тема, цель и исходная топология.
20. Список сетей и параметры course-net из docker network inspect (без секретов).
21. Команды создания сети и подключения pg-course.
22. Файлы app.js, Dockerfile, .dockerignore и структура каталога.
23. Результат сборки и запуска api-net6.
24. HTTP-ответ curl и результат SELECT COUNT(*) из PostgreSQL.
25. Описание двух диагностических экспериментов, ошибок и восстановления.
26. Ответы на контрольные вопросы, краткий вывод.

14. Контрольные вопросы

27. 1. Чем bridge отличается от host и none?
28. 2. Для чего нужна пользовательская сеть Docker?
29. 3. Как Docker разрешает имя pg-course в сети course-net?
30. 4. Почему localhost внутри API не указывает на PostgreSQL?

31. 5. Чем отличаются порты хоста и контейнера?
32. 6. Зачем используется -p 127.0.0.1:3000:3000?
33. 7. Почему не требуется публикация порта 5432 для API?
34. 8. Чем различаются docker network create и connect?
35. 9. Как определить сеть работающего контейнера?
36. 10. Как отличить ошибку Docker DNS от ошибки пароля PostgreSQL?
37. 11. Что произойдёт с данными PostgreSQL после docker stop?
38. 12. Какие шаги выполнить, если curl возвращает HTTP 503?

15. Критерии оценивания

Распределение баллов

Критерий	Баллы
Подготовка и проверка PostgreSQL	10
Создание и анализ Docker-сети	15
Подключение pg-course и проверка Docker DNS	15
Сборка образа Express API	20
Запуск и проверка HTTP/SQL	20
Диагностические эксперименты и восстановление	10
Отчёт и контрольные вопросы	10
Итого	100

После защиты оставьте pg-course, pgdata-course и course-net для следующих лабораторных. Контейнер api-net6 также можно сохранить. Не выполняйте docker volume prune или docker system prune с удалением томов на учебной машине.